

Séries temporais e sistemas de recomendação

Fabio Santiago
fabio.santiago@facens.br



Séries temporais: modelos clássicos

Cronograma

1. Conceito de sistema de recomendação
2. Filtragem baseada em conteúdo
3. Filtragem colaborativa
4. Filtragem colaborativa Baseada em memoria
5. Filtragem colaborativa Baseada em modelo
6. Fatoração de Matrizes
    - 6.1 Alternating Least Squares
    - 6.2 Singular Value Decomposition
    - 6.3 Stochastic Gradient Descent
7. Sistemas Híbridos

Conceito de Sistema de Recomendação

Conceito de Sistema de Recomendação

Um **Sistema de Recomendação (SR)** é um sistema de informação, geralmente baseado em técnicas de aprendizado de máquina e mineração de dados, cujo objetivo é **prever a preferência de um usuário por um item** (produto, filme, música, artigo, curso, etc.) e sugerir a ele itens que provavelmente sejam do seu interesse.







Conceito de Sistema de Recomendação

Conceito de Sistema de Recomendação

Formalmente, pode ser definido como uma função que estima a utilidade de um item i para um usuário u, expressa por:

$$\hat{r}_{u,i} = f(u, i)$$

Onde $\hat{r}_{u,i}$ é a predição da avaliação (rating) ou grau de interesse do usuário u pelo item i. O sistema então ordena os itens com maior valor previsto e recomenda os *top-N*.







Conceito de Sistema de Recomendação

Conceito de Sistema de Recomendação

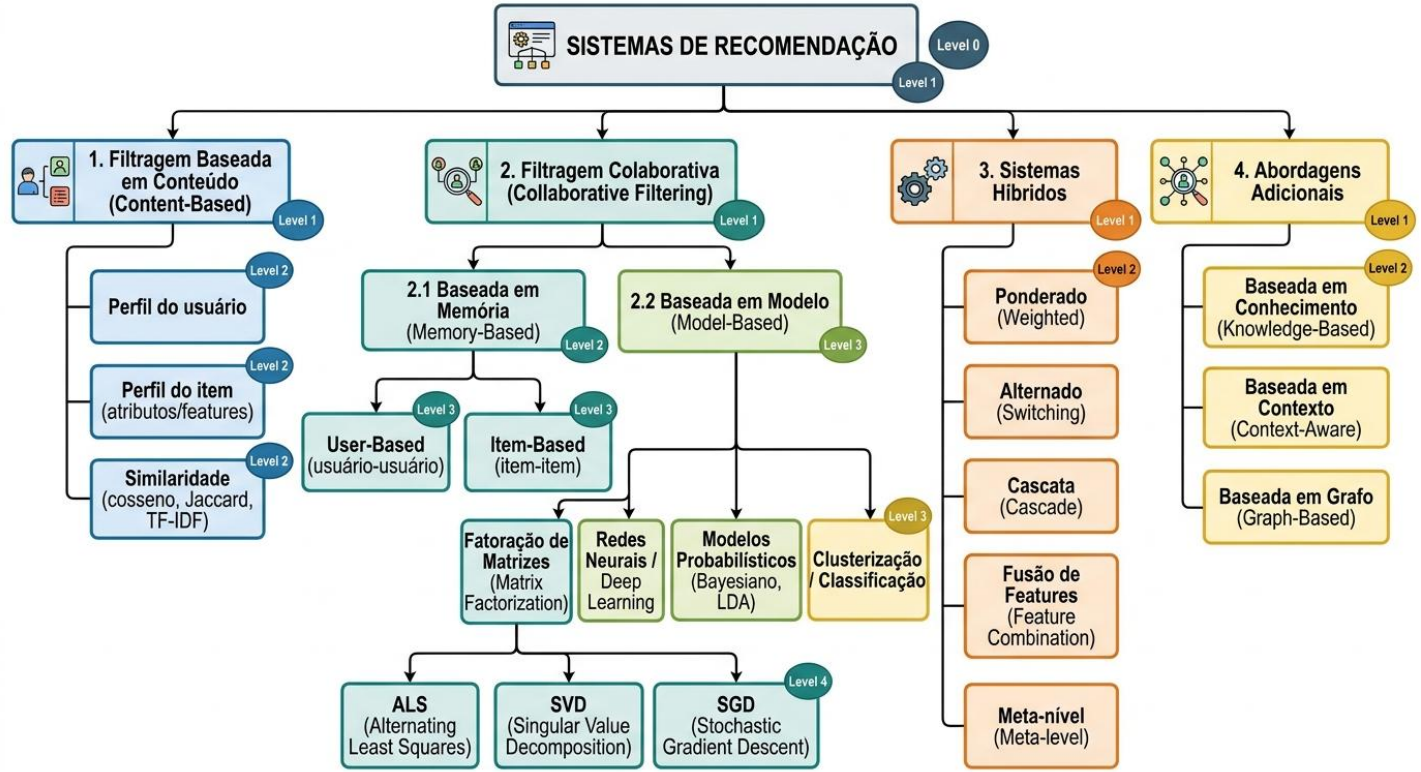


Aplicações típicas: Netflix (filmes), Amazon (produtos), Spotify (músicas), YouTube (vídeos), e-commerce, redes sociais, plataformas de ensino (recomendação de cursos, trilhas de aprendizagem, materiais complementares)





Conceito de Sistema de Recomendação



Filtragem baseada em conteúdo



Filtragem Baseada em Conteúdo (Content-Based Filtering)

Conceito: Recomenda itens **semelhantes** àqueles que o usuário já gostou no passado, com base nas **características (atributos) dos itens** e no **perfil do usuário**.





Filtragem baseada em conteúdo

Filtragem Baseada em Conteúdo (Content-Based Filtering)



Funcionamento:

- Cada item é representado por um vetor de features (ex.: gênero, diretor, palavras-chave, tags).
- O perfil do usuário é construído a partir dos itens que ele avaliou positivamente.
- Calcula-se a similaridade entre o perfil do usuário e cada item candidato (ex.: similaridade do cosseno).
- Recomendam-se os itens mais similares.





Filtragem baseada em conteúdo

Filtragem Baseada em Conteúdo (Content-Based Filtering)



Vantagens:

- Não sofre com o problema de *cold-start* de itens (novos itens podem ser descritos por suas features).
- Explicabilidade alta (fácil justificar a recomendação).
- Independente de outros usuários





Filtragem baseada em conteúdo

Filtragem Baseada em Conteúdo (Content-Based Filtering)



Desvantagens:

- **Sobre-especialização**: recomenda apenas itens similares ao histórico, limitando a diversidade.
- Difícil extrair boas features (especialmente em domínios como vídeo, áudio).
- Não captura preferências emergentes ou cruzadas entre usuários.





Filtragem baseada em conteúdo

Filtragem Baseada em Conteúdo (Content-Based Filtering)



Desvantagens:

- **Sobre-especialização**: recomenda apenas itens similares ao histórico, limitando a diversidade.
- Difícil extrair boas features (especialmente em domínios como vídeo, áudio).
- Não captura preferências emergentes ou cruzadas entre usuários.





Filtragem colaborativa

Filtragem Colaborativa (Collaborative Filtering — CF)



Conceito: Baseia-se na **sabedoria coletiva**. Não usa atributos dos itens, mas sim o **histórico de interações** (ratings, cliques, compras) de **muitos usuários** para inferir preferências.





Premissa: "Usuários que concordaram no passado tendem a concordar no futuro."

- .

Filtragem colaborativa

Filtragem Colaborativa (Collaborative Filtering — CF)



Desvantagens:

- **Cold-start**: difícil recomendar para novos usuários ou novos itens.
- **Sparsity**: matrizes de interação geralmente muito esparsas.
- **Scalability**: custo computacional cresce com o número de usuários/itens.
- **Gray sheep / Shilling attacks**: usuários com gostos atípicos ou manipulação maliciosa.





Filtragem colaborativa

Filtragem Colaborativa (Collaborative Filtering — CF)



Desvantagens:

- **Cold-start**: difícil recomendar para novos usuários ou novos itens.
- **Sparsity**: matrizes de interação geralmente muito esparsas.
- **Scalability**: custo computacional cresce com o número de usuários/itens.
- **Gray sheep / Shilling attacks**: usuários com gostos atípicos ou manipulação maliciosa.





Sistemas de recomendação

Filtragem Colaborativa (Collaborative Filtering — CF)

A CF se divide em duas grandes famílias: **baseada em memória** e **baseada em modelo**.







Filtragem colaborativa baseada em memória

Filtragem Colaborativa Baseada em Memória (Memory-Based)



Conceito: Utiliza **diretamente a matriz de interações** (user-item) para calcular similaridades e fazer predições, sem construir um modelo treinado. Também chamada de **Neighborhood-Based**.





Filtragem colaborativa baseada em memória

Filtragem Colaborativa Baseada em Memória (Memory-Based)



User-Based (Usuário-Usuário):

Encontra usuários **similares** ao usuário-alvo (vizinhos).

Prediz a nota para um item como média ponderada das notas dos vizinhos.





$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{v \in N(u)} s\,im(u,v) \cdot \left(r_{v,i} - \bar{r}_v\right)}{\sum_{v \in N(u)} |\,sim(u,v)\,|}$$

Filtragem colaborativa baseada em memória

Filtragem Colaborativa Baseada em Memória (Memory-Based)



Item-Based (Item-Item):

- Encontra itens **similares** ao item-alvo com base nas avaliações dos usuários.
- Prediz a nota usando a similaridade entre itens.
- Mais escalável que user-based (similaridades entre itens mudam menos).





Filtragem colaborativa baseada em memória

Filtragem Colaborativa Baseada em Memória (Memory-Based)



Métricas de similaridade:

- Cosseno
- Correlação de Pearson
- Jaccard (para dados binários)
- Cosseno ajustado.





Filtragem colaborativa baseada em memória

Filtragem Colaborativa Baseada em Memória (Memory-Based)

- **Vantagens:** simples, intuitivo, fácil de implementar e explicar.
- **Desvantagens:** não escala bem para grandes bases; sensível à esparsidade; precisa recalcular similaridades a cada nova interação.





Filtragem colaborativa baseada em modelo

Filtragem Colaborativa Baseada em Modelo (Model-Based)



Conceito: Constrói um **modelo preditivo** a partir dos dados históricos, que é então usado para gerar recomendações. O modelo **generaliza** padrões e lida melhor com esparsidade e escalabilidade.

- .





Filtragem colaborativa baseada em modelo

Filtragem Colaborativa Baseada em Modelo (Model-Based)



Técnicas comuns:

- Fatoração de Matrizes (Matrix Factorization)
- Redes Neurais / Deep Learning
- Modelos probabilísticos (Bayesiano, LDA)
- Clusterização (K-Means aplicado a usuários/itens)
- Máquinas de Vetores de Suporte (SVM)
- Modelos de Markov





Filtragem colaborativa baseada em modelo

Filtragem Colaborativa Baseada em Modelo (Model-Based)

Vantagens:

- Melhor desempenho preditivo.
- Escalável para grandes volumes.
- Lida melhor com esparsidade.
- Permite aprendizado incremental.







Filtragem colaborativa baseada em modelo

Filtragem Colaborativa Baseada em Modelo (Model-Based)



Desvantagens:

- Menor interpretabilidade ("caixa-preta").
- Custo de treinamento.
- Necessita dados suficientes para treinar.





Sistemas de recomendação

Matrix Factorization (Fatoração de Matrizes)



Conceito Central: A ideia é **decompor a matriz de ratings** $R \in \mathbb{R}^{m \times n}$(m usuários × n itens) em **duas matrizes de fatores latentes** de dimensão reduzida k:

$$R \approx P \cdot Q^T$$





Onde:

- $P \in \mathbb{R}^{m \times k} \rightarrow$ matriz de fatores latentes dos **usuários**
- $Q \in \mathbb{R}^{n \times k} \rightarrow$ matriz de fatores latentes dos **itens**
- $k \ll \min(m, n) \rightarrow$ dimensão latente (ex.: 20, 50, 100)

Filtragem colaborativa baseada em modelo

Matrix Factorization (Fatoração de Matrizes)



Cada usuário e item é representado por um **vetor latente** em um espaço de dimensão k. A predição é o **produto interno**:

$$\hat{r}_{u,i} = p_u^T q_i$$





Os fatores latentes capturam conceitos implícitos (ex.: em filmes: "ação", "romance", "drama"; em cursos: "programação", "matemática", "gestão").

ALS (Alternating Least Squares)

Matrix Factorization (Fatoração de Matrizes)



ALS (Alternating Least Squares)

Conceito: Técnica de otimização para fatoração de matrizes que **alterna** entre fixar P e resolver Q, e vice-versa, usando **mínimos quadrados**.





ALS (Alternating Least Squares)

ALS (Alternating Least Squares)



Equação (passo para p_u):

$$p_u = (Q^T Q + \lambda I)^{-1} Q^T r_u$$





Onde λé o parâmetro de regularização (evita overfitting).

ALS (Alternating Least Squares)

ALS (Alternating Least Squares)



Funcionamento:

1. Inicializa Pe Qaleatoriamente.
2. Fixa Qe resolve Panaliticamente (solução de forma fechada).
3. Fixa Pe resolve Qanaliticamente.
4. Repete até convergir.





ALS (Alternating Least Squares)

ALS (Alternating Least Squares)



Vantagens:

- **Paralelizável** (cada linha pode ser resolvida independentemente).
- Bom para **feedback implícito** (ex.: cliques, visualizações).
- Convergência estável.





ALS (Alternating Least Squares)

ALS (Alternating Least Squares)



Desvantagens:

Cada iteração resolve sistemas lineares (custo $O(k^3)$por usuário/item).

Não lida naturalmente com dados faltantes de forma incremental.

Uso típico: Spark MLlib (ALS é o algoritmo padrão de CF no Apache Spark)





SVD (Singular Value Decomposition)

SVD (Singular Value Decomposition)

Conceito: Decomposição matricial clássica da álgebra linear. Para uma matriz R:

$$R = U\Sigma V^T$$







Onde:

- U: vetores singulares à esquerda (usuários)
- Σ: matriz diagonal de valores singulares
- V^T: vetores singulares à direita (itens)

SVD (Singular Value Decomposition)

SVD (Singular Value Decomposition)

No contexto de SR: usa-se **SVD truncado** (apenas os k maiores valores singulares) para reduzir dimensionalidade e capturar a estrutura latente.







SVD (Singular Value Decomposition)

SVD (Singular Value Decomposition)



Variantes importantes:

- **FunkSVD**: proposta por Simon Funk no Netflix Prize; aprende Pe Qpor gradiente descendente (não é SVD matemático puro).
- **SVD++**: incorpora feedback implícito (o que o usuário viu/comprou, mesmo sem avaliar).
- **Truncated SVD**: usado em LSA (Latent Semantic Analysis)





SVD (Singular Value Decomposition)

SVD (Singular Value Decomposition)



Variantes importantes:

- Base matemática sólida.
- Boa redução de dimensionalidade.
- Captura relações latentes.





Sistemas de recomendação

SGD (Stochastic Gradient Descent)

Conceito: Método iterativo de otimização que atualiza os parâmetros **um exemplo por vez** (ou mini-batch), seguindo o gradiente negativo da função de erro.







SGD (Stochastic Gradient Descent)

SGD (Stochastic Gradient Descent)

Função de erro (com regularização):



$$E = \sum_{(u,i)\in\mathcal{K}} (r_{u,i} - p_u^T q_i)^2 + \lambda(\| p_u \|^2 + \| q_i \|^2)$$





Atualização:

$$p_u \leftarrow p_u + \eta \cdot (e_{u,i} \cdot q_i - \lambda \cdot p_u)$$

$$q_i \leftarrow q_i + \eta \cdot (e_{u,i} \cdot p_u - \lambda \cdot q_i)$$

SGD (Stochastic Gradient Descent)

SGD (Stochastic Gradient Descent)

Onde:

- $e_{u,i} = r_{u,i} - \hat{r}_{u,i} \rightarrow$ erro de predição
- $\eta \rightarrow$ taxa de aprendizado (learning rate)
- $\lambda \rightarrow$ regularização





SGD (Stochastic Gradient Descent)

SGD (Stochastic Gradient Descent)



Vantagens:

- **Simples** de implementar.
- **Escalável** para grandes bases.
- Convergência rápida em prática.
- Base do **FunkSVD** e de muitos modelos modernos.





SGD (Stochastic Gradient Descent)

SGD (Stochastic Gradient Descent)



Desvantagens:

- Sensível à escolha de η e λ.
- Pode convergir para mínimos locais.
- Requer várias épocas (epochs) sobre os dados.
- **Uso típico:** implementações em Python (Surprise, TensorFlow, PyTorch) e no Netflix Prize.





Sistemas híbridos

Conceito: Combinam **duas ou mais técnicas** de recomendação para superar as limitações individuais de cada abordagem (ex.: cold-start da CF, sobre-especialização do content-based).







Sistemas híbridos

Principais estratégias:

Estratégia	Descrição
Ponderada (Weighted)	Combina scores de vários recomendadores com pesos.
Alternada (Switching)	Escolhe qual recomendador usar conforme a situação (ex.: novo usuário → content-based).
Cascata (Cascade)	Um recomendador refina a saída de outro.







Sistemas híbridos

Principais estratégias:

Fusão de Features (Feature Combination)	Combina features de conteúdo com features colaborativas em um único modelo.
Meta-nível (Meta-level)	Um modelo aprende a partir da saída de outro.
Mista (Mixed)	Apresenta recomendações de várias fontes simultaneamente.







Sistemas híbridos

Principais estratégias:

Exemplo prático em educação:



- Para **novos alunos** (cold-start): usar content-based com base no curso matriculado.





- Para **alunos com histórico**: usar CF (ALS/SGD) com base em avaliações de outros alunos.

- Combinar ambos com pesos ajustados dinamicamente

Sistemas híbridos

Sistemas Híbridos

- **Vantagens:**
- Mitiga cold-start e esparsidade.
- Maior acurácia e diversidade.
- Mais robusto





Sistemas híbridos

Desvantagens:

- Maior complexidade de implementação e manutenção.
- Custo computacional elevado.
- Difícil calibrar pesos.







Sistemas de recomendação

BIBLIOGRAFIA BÁSICA

HYNDMAN, R. J.; ATHANASOPOULOS, G. Forecasting: Principles and Practice. 3. ed. Melbourne: OTexts, 2021.

BOX, G. E. P.; JENKINS, G. M.; REINSEL, G. C.; LJUNG, G. M. Time Series Analysis: Forecasting and Control. 5. ed. Hoboken: Wiley, 2015.

AGGARWAL, C. C. Recommender Systems: The Textbook. Cham: Springer, 2016. 6.

BIBLIOGRAFIA COMPLEMENTAR

RICCI, F.; ROKACH, L.; SHAPIRA, B. Recommender Systems Handbook. 2. ed. Cham: Springer, 2022. • GRUS, J. Data Science do Zero. 2. ed. Rio de Janeiro: Alta Books, 2020.

ALMEIDA, J. M. et al. Sistemas de Recomendação. Rio de Janeiro: LTC, 2016. • MORETTIN, P. A.; TOLOI, C. M. C. Análise de Séries Temporais. 3. ed. São Paulo: Blucher, 2018.

BROWNLEE, J. Deep Learning for Time Series Forecasting. Melbourne: Machine Learning Mastery, 2018.


Facens
AQUI TEM ENGENHARIA